

=====
SSCEVS – Student Council Election Voting System
Documentation
=====

OVERVIEW

SSCEVS (Student Council Election Voting System) is a web application for managing student council elections at an educational institution. It covers the full election lifecycle: voter registration with ID verification, admin review, secure online voting, live monitoring, ballot receipts, results, and support.

The system is built for BCC-style (Baao Community College / similar Philippine college) student IDs, with OCR tuned for common ID layouts and student ID formats (e.g. 23-1-0516).

TECH STACK

Table with 2 columns: Layer, Technology. Rows include Backend (Laravel 12), Frontend (Vue 3), Styling (Tailwind CSS), Build (Vite), Database (MySQL), Queue/Cache/Sessions (Database drivers), Real-time (Laravel Echo), PDF (DomPDF), OCR (OCR.Space API), AI FAQ (Groq API), Charts (Unovis), Testing (Pest PHP).

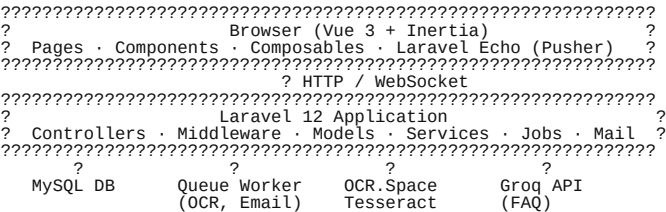
USER ROLES

- VOTER
Students who register, get verified, and cast ballots.
- ADMIN
Election administrators with full management access. Admin emails use the domain @sscevs.admin.com (e.g. admin@sscevs.admin.com).
Role is determined at registration/login from the email domain:
- Emails ending in @sscevs.admin.com ? admin
- All other emails ? voter

CORE FEATURES

- FOR VOTERS
- Multi-step registration (profile ? ID scan ? OTP ? admin/auto verification)
- Elections page with candidate cards per position
- One ballot per election (one vote per position)
- Ballot receipt with PDF download (signed URL)
- My Votes, Results, Announcements
- Help & Support tickets with live chat (after admin approval)
- AI-powered FAQ chat (SSCEVS-scoped)
- Profile photo and password management
- Public registration status check (no login required)
- FOR ADMINS
- Election CRUD (draft ? scheduled ? active ? closed)
- Candidate management (photo, platform, position, department, partylist)
- Voter list with OCR data, fraud score, and verify/reject actions
- Live election monitoring (turnout, standings, analytics)
- Support ticket approval and chat
- Settings: departments, courses, year levels, positions, partylists
- Admin account management
- Reports page

ARCHITECTURE



KEY DIRECTORIES

Table with 2 columns: Directory, Contents. Rows include app/ (Route handlers, Admin guard, etc.), resources/js/ (Inertia page components, UI, etc.), database/migrations/ (Schema definitions), database/seeder/ (Default admin, sample election data), routes/web.php (All web routes).

DATABASE MODELS

Model	Purpose
-----	-----
User	Voters and admins; profile, OCR fields, fraud score
Election	Election metadata, event/voting windows, status
Candidate	Candidates linked to election, position, department
Vote	Individual vote records (user + election + position + candidate)
BallotReceipt	Proof of ballot submission (BR-YYYY-XXXXXX)
Department	Academic departments (name, acronym, color)
Course	Courses under departments (with duration_years)
YearLevel	1st-4th year (with sort_order)
Position	Officer positions (President, VP, etc.)
Partylist	Party/group affiliations for candidates
RegistrationAttempt	Device fingerprint audit trail for fraud detection
SupportTicket	Help desk tickets
SupportMessage	Chat messages on approved tickets

ELECTION STATUSES

```

draft      ? Not yet published
scheduled  ? Published, voting may not have started
active     ? Election is live
closed     ? Election finished

```

Voting is open when status is active or scheduled AND the current time is within voting_starts_at and voting_ends_at.

REGISTRATION FLOW

```

Step 1: Register form
? Name, email, student ID, department, course, year level, password
? Data stored in session (no user created yet)

Step 2: ID Scan
? Capture/upload school ID photo (client-side quality check: good/warn/blurry)
? User account created with voter ID (VID-YYYY-XXXXX)
? Verification email + OTP sent (queued)

Step 3: OTP verification
? Voter enters 6-digit OTP (or clicks email verification link)
? Email marked verified; OCR job queued

Step 4: Background OCR + fraud scoring
? ID image processed (OCR.Space ? Tesseract fallback)
? Fraud score calculated from OCR match, duplicates, device fingerprint, etc.

Step 5: Auto or manual verification
? Score >= 80: auto-approved (is_verified = true)
? Score < 80: pending admin review on /voters page
? Admin can Approve, Reject, or Re-run OCR

```

REGISTRATION FLOW DIAGRAM

```

????????????????????
?   Guest visits      ?
?   /register         ?
????????????????????
?
?
????????????????????
? STEP 1 – Registration Form
? Fields: name, email, student ID, department, course, year level, password?
? Validation: unique email & student ID, year level fits course duration ?
? Output: data saved to session (reg_step1) – no user record yet
????????????????????
?
?
????????????????????
? STEP 2 – ID Scan (/register/id-scan)
? • Capture/upload school ID photo
? • Client checks image quality: good / warn / blurry
? • User account created (role: voter)
? • Voter ID assigned: VID-YYYY-XXXXX
? • ID image stored; device fingerprint logged
? • SendVerificationEmail job queued (OTP + verify link)
????????????????????
?
?
????????????????????
? Registration      ?
? Success page      ?
????????????????
?
?
????????????????????
? STEP 3 – OTP Verification (/register/verify-otp)
?
?   ?????????????????????   ?????????????????????
?   ? Enter 6-digit OTP ? OR ? Click email link ?
?   ? on verify-otp page ? ? /email/verify/{token}?
?   ?????????????????????   ?????????????????????
?
?   ?
?   ?????????????????????
?
?   email_verified_at set · registration_status = active
?   ProcessOcrVerification job dispatched
????????????????????
?
?
????????????????????
? STEP 4 – Background Job (ProcessOcrVerification)
? Queue worker required
?
?   ?????????????????????   ?????????????????????   ?????????????????????
?   ? Load ID   ?????? OCR.Space API   ?????? Extract name,   ?
?   ? image     ? ? (Tesseract if ? ? student ID, course ?
?   ????????????????????? ? API fails) ? ?????????????????????
?   ?   ?????????????????????? ?
?   ?   ?
?   ?   ?????????????????????
?   ?   ? FraudDetectionService ?
?   ?   ? calculates fraud_score ?
?   ?   ?????????????????????

```


[illegible]

- MySQL
- (Optional) Tesseract OCR for local fallback
- (Optional) Pusher account for real-time chat

Quick setup:
composer setup

This runs: composer install, copies .env, generates app key, migrates, npm install, and npm run build.

Manual setup:
composer install
cp .env.example .env
php artisan key:generate

Configure DB_* in .env, then:
php artisan migrate
php artisan db:seed
php artisan storage:link

npm install
npm run build

Development:
composer dev

- Starts concurrently:
- php artisan serve
 - php artisan queue:listen
 - npm run dev (Vite HMR)

The queue worker is REQUIRED for OCR processing and verification emails.

ENVIRONMENT VARIABLES

Variable	Purpose
DB_*	MySQL connection (DB_DATABASE=sscevs)
QUEUE_CONNECTION	database (for OCR/email jobs)
SESSION_DRIVER	database
MAIL_*	Email delivery (OTP, verification)
OCRSPACE_API_KEY	OCR.Space API key
GROQ_API_KEY	FAQ chat AI
GROQ_MODEL	Default: llama-3.3-70b-versatile
PUSHER_*	Real-time broadcasting
AWS_*	Optional S3 storage

DEFAULT SEEDED ACCOUNTS

After php artisan db:seed:

Role	Email	Password
Admin	admin@sscevs.admin.com	Admin@1234
Voter	juan@example.com	Voter@1234

Also seeds: year levels, positions, a sample department/course, and a scheduled 2026 election with sample candidates.

ROUTE MAP

PUBLIC (GUEST)		
GET /		Welcome page
GET /check-status		Check registration status form
POST /check-status		Submit status check
GET /register		Voter registration form
POST /register		Submit registration step 1
GET /register/id-scan		ID photo capture
POST /register/id-scan		Submit ID scan
GET /register/success		Registration success page
GET /register/verify-otp		OTP verification form
POST /register/verify-otp		Submit OTP
POST /register/resend-otp		Resend OTP
GET /email/verify/{token}		Email verification link
GET /login		Login form
POST /login		Submit login
PUBLIC (SIGNED URL)		
GET /ballot-receipt/{receipt}/pdf		Download ballot receipt PDF
AUTHENTICATED (ALL ROLES)		
GET /registration-status		Registration status page
GET /dashboard		Dashboard
GET /profile		User profile
POST /profile/photo		Update profile photo
POST /profile/password		Update password
GET /elections		Elections (admin: manage, voter: vote)
POST /elections/{election}/cast-vote		Submit ballot
GET /ballot-receipt/{receipt}		View ballot receipt
GET /my-votes		Submitted ballots
GET /results		Election results
GET /announcements		Announcements
GET /help		Help & Support
POST /help/tickets		Create support ticket
GET /help/tickets/{ticket}		View support ticket
POST /help/tickets/{ticket}/messages		Send support message
GET /faq		FAQ page
POST /faq/chat		FAQ AI chat (throttled)
GET /voters		Voter list
GET /reports		Reports
POST /logout		Logout
ADMIN ONLY (admin middleware)		
POST /elections		Create election
PUT /elections/{election}		Update election
DELETE /elections/{election}		Delete election
GET /candidates		Candidate list
GET /candidates/create		Create candidate form
POST /candidates		Store candidate
GET /candidates/{candidate}/edit		Edit candidate form
PUT /candidates/{candidate}		Update candidate
DELETE /candidates/{candidate}		Delete candidate
GET /voters/{voter}		Voter detail

```

POST /voters/{voter}/verify      Approve voter
POST /voters/{voter}/reject      Reject voter
POST /voters/{voter}/rerun-ocr   Re-run OCR on voter ID
GET /support                     Support ticket management
GET /support/tickets/{ticket}    Admin view ticket
POST /support/tickets/{ticket}/approve Approve ticket
POST /support/tickets/{ticket}/reject Reject ticket
POST /support/tickets/{ticket}/close Close ticket
POST /support/tickets/{ticket}/messages Admin send message
GET /monitoring                  Live election monitoring
GET /settings                    System settings
POST /settings/departments       Create department
PUT /settings/departments/{department} Update department
DELETE /settings/departments/{department} Delete department
POST /settings/courses           Create course
PUT /settings/courses/{course}   Update course
DELETE /settings/courses/{course} Delete course
POST /settings/year-levels       Create year level
PUT /settings/year-levels/{yearLevel} Update year level
DELETE /settings/year-levels/{yearLevel} Delete year level
POST /settings/positions         Create position
PUT /settings/positions/{position} Update position
DELETE /settings/positions/{position} Delete position
POST /settings/partylists        Create partylist
PUT /settings/partylists/{partylist} Update partylist
DELETE /settings/partylists/{partylist} Delete partylist
GET /accounts                    Admin account list
POST /accounts                   Create admin account
PUT /accounts/{user}             Update admin account
DELETE /accounts/{user}          Delete admin account

```

SECURITY FEATURES

- Password hashing (bcrypt)
- Signed URLs for ballot receipt PDFs
- Admin middleware on sensitive routes
- OTP with attempt tracking
- Device fingerprint logging for registration fraud
- Duplicate email/student ID detection
- Vote immutability (one ballot per election)
- FAQ chat rate limiting (15 requests per minute)
- Admin emails restricted to @sscevs.admin.com domain

TESTING

```

composer test
# or
php artisan test

```

Uses Pest PHP with the Laravel plugin.

THIRD-PARTY DEPENDENCIES

Package	Use
inertiajs/inertia-laravel	SPA-like navigation without separate API
barryvdh/laravel-dompdf	Ballot receipt PDFs
aws/aws-sdk-php	Optional cloud storage
pusher/pusher-php-server	Server-side broadcasting
tesseract.js	Client-side OCR assist on ID scan page
@unovis/vue	Monitoring/dashboard charts
vue-sonner	Toast notifications

TYPICAL ADMIN WORKFLOW

1. Configure Settings (departments, courses, year levels, positions, partylists).
2. Create an Election and set voting window.
3. Add Candidates for each position.
4. Set election status to Active when voting should open.
5. Review pending Voters (check fraud score and OCR data).
6. Monitor turnout on Monitoring during voting.
7. Close election and publish Results.

TYPICAL VOTER WORKFLOW

1. Register with student details.
2. Scan/upload school ID.
3. Verify OTP from email.
4. Wait for account verification (auto or admin).
5. Cast vote during the Elections window.
6. Download ballot receipt as proof.
7. View Results after the voting period ends.

SUPPORT TICKET STATUSES

```

pending    ? Awaiting admin approval
approved   ? Live chat enabled
rejected   ? Ticket denied
closed     ? Ticket resolved

```

USER REGISTRATION STATUSES

```

active      ? Fully registered and verified
pending_id_scan ? Awaiting ID scan (legacy)
pending_otp  ? Awaiting OTP verification

```

BALLOT RECEIPT FORMAT

Receipt numbers: BR-YYYY-XXXXXX (e.g. BR-2026-000001)
Voter ID numbers: VID-YYYY-XXXXX (e.g. VID-2026-00001)

=====

